



API Contract (api-contract.md) — Orvio

Version: 0.1 (MVP-ready)

Last updated: 2026-09-28

Owner: Engineering (Code X)

Applies to: Frontend, Backend, QA, AI coding agents

1. Overview

This document defines the **formal HTTP API contract** between Orvio clients (React web app, POS client) and the Orvio backend (Node.js + Fastify). It specifies endpoints, request/response shapes, authentication, authorization, error handling, pagination, and multi-tenant boundaries.

Goals:

- Enable frontend developers to implement API clients without guessing backend behavior.
- Enable backend developers to implement endpoints without guessing frontend expectations.
- Provide AI coding agents a reliable contract to modify safely.
- Ensure consistent, secure, and backward-compatible API evolution.

Scope: MVP modules — Auth, Org, Inventory, POS, Orders (WhatsApp), Payments, Reports, Settings.

2. API Principles

- **RESTful**, resource-oriented, predictable URLs.
- **Stateless**, JWT-based authentication.
- **Multi-tenant**: every request scoped to an organization (`org_id`).
- **Consistent** response envelope (data, meta, error).
- **Strong validation** at API boundary; business rules enforced server-side.
- **Backward compatible**: breaking changes require version bump or deprecation.
- **Secure**: HTTPS, CORS, rate limiting, input validation, tenant isolation.

3. Source of Truth

When determining API behavior, use this priority:

1. Explicit project requirements
2. PRD (`docs/PRD.md`)
3. Architecture (`architecture.md`)
4. Existing API implementation

5. Existing frontend consumers
6. Existing tests
7. Existing documentation
8. Reasonable API design conventions

Do not invent endpoints. Every endpoint must trace to a product requirement or existing capability.

4. Base URL

Pattern: `/api/v1`

Environment URLs (placeholders):

- **Development:** `http://localhost:3000/api/v1`
- **Staging:** `https://staging-api.orvio.africa/api/v1`
- **Production:** `https://api.orvio.africa/api/v1`

(Actual domains to be configured in deployment.)

5. Versioning

Strategy: URL versioning (`/api/v1`, `/api/v2`, ...).

Breaking changes (require new version or deprecation):

- Removing/renaming fields
- Changing field types
- Removing endpoints
- Changing authentication/authorization requirements
- Changing response semantics

Non-breaking changes:

- Adding optional fields
- Adding new endpoints
- Adding optional query parameters

Deprecation process:

1. Announce deprecation (docs, changelog).
2. Provide migration guide.
3. Sunset after ≥ 90 days.

6. Naming Conventions

- **URLs:** Lowercase, kebab-case for multi-word resources.
 - `/api/v1/products`, `/api/v1/stock-adjustments`
- **JSON fields:** camelCase (e.g., `createdAt`, `orgId`).
- **Do not mix** snake_case/camelCase without documented reason.

7. Authentication

Scheme: JWT Bearer token.

Header: Authorization: Bearer <access_token>

Token types:

- **Access token:** Short-lived (15 min), in memory.
- **Refresh token:** Longer (7–30 days), httpOnly cookie or secure storage.

Endpoints:

- POST /auth/login
- POST /auth/register
- POST /auth/refresh
- POST /auth/logout
- POST /auth/reset-password

Security:

- HTTPS only.
- Token rotation on refresh.
- Revocation on suspicious activity.

8. Authorization

Model: Role-based access control (RBAC) with organization/branch scoping.

Roles: Owner, Manager, Cashier, Storekeeper, Accountant, Internal Admin.

Permissions: Evaluated server-side per request (e.g., `inventory.products.create`, `pos.sales.void`).

Frontend permissions: Advisory only (UX). Backend authorization is the security boundary.

Response: Optional `permissions` array in `/me` endpoint for UI gating.

9. Multi-Tenancy

Tenant = Organization. Every request must include `org_id` (from token or validated body).

Isolation: All queries scoped by `org_id`; repository enforces.

Never trust: Client-provided `org_id` without authorization.

Cross-tenant access: Explicitly prohibited; tested via isolation tests.

10. HTTP Methods

Method	Purpose	Typical Behavior
GET	Retrieve	Read resource(s)
POST	Create/action	Create resource or explicit operation
PATCH	Partial update	Modify selected fields
PUT	Full replacement	Replace resource
DELETE	Remove	Delete/archive resource

Actions (non-CRUD): Use POST with clear path (e.g., /stock/adjustments).

11. Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request (validation)
401	Unauthorized (not authenticated)
403	Forbidden (authenticated, no permission)
404	Not Found
409	Conflict
422	Unprocessable Entity (business rule violation)
429	Too Many Requests (rate limit)
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

Distinction:

- 401: Not authenticated.
- 403: Authenticated, lacks permission.
- 404: Resource does not exist (or hidden for security).
- 409: Conflict with current state.
- 422: Valid request, violates business rules.

12. Error Contract

Format:

```
{
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "The requested resource was not found.",
    "details": null,
    "requestId": "req_123"
  }
}
```

Fields:

- `code`: Machine-readable error code.
- `message`: User-friendly message.
- `details`: Optional structured details (e.g., validation errors).
- `requestId`: Correlation ID for logs.

Standard error codes:

- AUTHENTICATION_REQUIRED
- INVALID_CREDENTIALS
- SESSION_EXPIRED
- FORBIDDEN
- RESOURCE_NOT_FOUND
- VALIDATION_ERROR
- RESOURCE_CONFLICT
- RATE_LIMITED
- INVALID_REQUEST
- DEPENDENCY_FAILURE
- INTERNAL_ERROR

13. Request Conventions

Headers:

- `Authorization`: Bearer <token> (authenticated endpoints)
- `Content-Type`: application/json
- `Idempotency-Key`: <uuid> (for idempotent POSTs)

Body: JSON, camelCase fields.

Validation: Required/optional, string length, enums, numeric ranges, cross-field rules.

14. Response Conventions

Envelope:

- **Single resource:**

```
{
  "data": { "id": "123", "name": "Example" },
  "meta": { "requestId": "req_123" }
}
```

- **Collection:**

```
{
  "data": [ ... ],
  "pagination": { "nextCursor": "abc", "hasMore": true },
  "meta": { "requestId": "req_123" }
}
```

Consistency: All endpoints use this envelope.

15. Pagination

Strategy: Cursor-based pagination (preferred for large datasets).

Query params: ?cursor=abc&limit=25

Response:

```
{
  "data": [ ... ],
  "pagination": {
    "nextCursor": "abc",
    "hasMore": true
  }
}
```

Defaults: limit=25, max limit=100.

16. Filtering

Convention: Query params (e.g., ?status=active&branchId=xyz).

Supported filters: Whitelisted fields per endpoint.

Authorization: Filters respect tenant/branch scoping.

Invalid filters: Return 400 `VALIDATION_ERROR`.

17. Sorting

Convention: ?sort=createdAt (asc), ?sort=-createdAt (desc).

Allowed fields: Whitelisted per endpoint.

Default: Resource-specific (e.g., createdAt desc).

Invalid fields: 400 VALIDATION_ERROR.

18. Search

Parameter: ?q=...

Searchable fields: Whitelisted (e.g., product name, SKU, barcode).

Behavior: Case-insensitive partial match.

Min length: 2 chars.

Special chars: Sanitized; no SQL exposure.

19. Bulk Operations

MVP: Not included.

Future: If added, define:

- Max batch size (e.g., 100).
- Atomic vs partial failure.
- Transaction behavior.
- Idempotency.

20. Idempotency

Required for: Payments, sales, invitations.

Header: Idempotency-Key: <uuid>

Retention: 24 hours.

Duplicate behavior: Return original response.

21. Concurrency

At-risk operations: Stock updates, sales.

Strategy: Database transactions + optimistic concurrency (version field if needed).

Conflict response: 409 RESOURCE_CONFLICT.

22. Rate Limiting

Categories:

- Auth (login, reset): Strict.
- Public endpoints: Moderate.
- General API: Standard.
- Sensitive ops: Strict.

Limits: OPEN DECISION (configure per env).

Response: 429 RATE_LIMITED with Retry-After header.

23. Caching

Client cache: Cache-Control for static assets.

API cache: TanStack Query on frontend.

Server cache: In-memory config; no Redis in MVP.

Sensitive data: Never cache private responses.

24. File Uploads

MVP: Minimal (product images optional).

Flow: Direct to S3-compatible storage via signed URL.

Validation: MIME, size (e.g., $\leq 5\text{MB}$), virus scan (future).

Auth: Required.

25. Webhooks

Direction: Inbound (Paystack, WhatsApp BSP).

Endpoints:

- POST /webhooks/paystack
- POST /webhooks/whatsapp

Auth: HMAC signature verification.

Payload: Provider-specific; normalized internally.

Retry: Provider-defined; idempotent handling.

26. Real-Time APIs

MVP: Polling (30–60s) for orders/notifications.

Future: WebSockets/SSE for live dashboards.

27. Events

Internal events (not exposed):

- `user.created`
- `order.completed`
- `payment.succeeded`

Delivery: Internal queue (future); not part of public API.

28. Audit APIs

MVP: Internal audit logs (not exposed).

Future: `GET /audit-logs` for Owner/Manager with filtering/pagination.

29. API Security

- **HTTPS:** Enforced.
- **CORS:** Configured per env (no * for authed APIs).
- **CSRF:** Not applicable (JWT, no cookies for auth in MVP).
- **Input validation:** All requests.
- **Output encoding:** Prevent XSS.
- **Rate limiting:** Per category.
- **Request size:** Limit (e.g., 1MB).
- **Webhook verification:** HMAC.
- **Secrets:** Never exposed.

30. Date/Time

Format: ISO 8601, UTC.

Example: `2026-09-28T22:30:00Z`

Date-only: `YYYY-MM-DD`.

31. Money/Currency

Format: Minor units (integer), currency code.

Example:

```
{
  "amount": 250000,
  "currency": "NGN"
}
```

Precision: 2 decimals (NGN kobo).

Rounding: Half-up.

32. Enums/Statuses

Examples:

- **UserStatus:** active, suspended, pending, deleted.
- **OrderStatus:** draft, pending_payment, paid, fulfilled, refunded, cancelled.
- **PaymentStatus:** pending, succeeded, failed, refunded.

Transitions: Defined per resource (state machine).

33. Request IDs

Header: X-Request-ID: req_123

Generation: Server (if not provided).

Propagation: Logs, error responses.

34. Observability

Metrics:

- Request latency
- Error rate
- Status code distribution
- Auth failures
- Rate-limit events
- Dependency failures

Logs: Structured (Pino), include requestId, orgId, userId.

Do not expose: Internal monitoring data.

35. Deprecation

Process:

1. Announce (docs, changelog).
2. Migration guide.
3. Sunset ≥ 90 days.

Versioning: Breaking changes → new version.

36. OpenAPI / Swagger

Decision: Generate OpenAPI from Fastify schemas (source of truth: code).

Location: apps/api/docs/openapi.yaml (future).

37. Frontend Consumption Rules

- **API client:** apps/web/src/services/.
- **Auth:** Attach JWT to Authorization header.
- **Errors:** Use centralized error handler.
- **Caching:** TanStack Query (cache keys include orgId).
- **Mutations:** Optimistic updates where safe.
- **Retry:** Exponential backoff for 5xx/429.
- **Cancellation:** AbortController on unmount.

38. Backend Rules

- Validate all input.
- Authenticate & authorize.
- Enforce business rules.
- Enforce tenant isolation.
- Normalize errors.
- Maintain contract consistency.
- Avoid exposing internals.

39. Contract Testing

- **Backend:** Schema, request/response validation, status codes, errors.
- **Frontend:** API client tests, response parsing, error handling.
- **Integration:** E2E contract tests.
- **OpenAPI:** Auto-generate from schemas.

40. AI Agent Rules

AI agents must:

- Search for existing endpoints before creating.
- Search all consumers before changing.
- Verify no active consumers before deleting.
- Check for reusable schemas.

- Review auth lifecycle before changing auth.
- Review every affected role before changing authorization.
- Review affected API behavior before changing DB.

41. API Invariants

- Every protected endpoint requires authentication.
- Authorization is enforced server-side.
- API errors use the standard error schema.
- Dates use ISO 8601.
- Collection endpoints use standard pagination.
- Tenant-scoped resources cannot cross tenant boundaries.
- Sensitive fields are never returned unnecessarily.
- Breaking changes require versioning or explicit migration.

42. Complete Endpoint Reference

Auth

ID	Method	Endpoint	Purpose	Auth	Permission
AUTH-001	POST	/auth/register	Register user	No	Public
AUTH-002	POST	/auth/login	Login	No	Public
AUTH-003	POST	/auth/refresh	Refresh token	No	Public
AUTH-004	POST	/auth/logout	Logout	Yes	Authenticated
AUTH-005	POST	/auth/reset-password	Request reset	No	Public

AUTH-002: POST /auth/login

Purpose: Authenticate user, issue tokens.

Request:

```
{
  "email": "user@example.com",
  "password": "secret"
}
```

Success (200):

```
{
  "data": {
    "accessToken": "...",
    "refreshToken": "...",
    "user": { "id": "u1", "email": "user@example.com" }
  },
}
```

```
"meta": { "requestId": "req_123" }
}
```

Errors: 401 INVALID_CREDENTIALS, 400 VALIDATION_ERROR.

Side effects: Audit log (login attempt).

Idempotency: No.

Organization

ID	Method	Endpoint	Purpose	Auth	Permission
ORG-001	GET	/orgs/:id	Get org	Yes	Member
ORG-002	PATCH	/orgs/:id	Update org	Yes	Owner

Inventory

ID	Method	Endpoint	Purpose	Auth	Permission
INV-001	GET	/products	List products	Yes	inventory.products.read
INV-002	POST	/products	Create product	Yes	inventory.products.create
INV-003	GET	/products/:id	Get product	Yes	inventory.products.read
INV-004	PATCH	/products/:id	Update product	Yes	inventory.products.update
INV-005	POST	/stock/adjustments	Adjust stock	Yes	inventory.stock.adjust
INV-006	POST	/stock/transfers	Transfer stock	Yes	inventory.stock.transfer

INV-005: POST /stock/adjustments

Purpose: Record stock adjustment (damage, loss, found).

Request:

```
{
  "branchId": "b1",
  "productId": "p1",
  "quantity": -2,
  "reason": "damage",
  "notes": "Broken during handling"
}
```

Success (201):

```
{
  "data": {
    "id": "adj1",
    "productId": "p1",
    "branchId": "b1",
    "quantity": -2,
    "reason": "damage",

```

```
    "createdAt": "2026-09-28T22:30:00Z"
  },
  "meta": { "requestId": "req_123" }
}
```

Errors: 400 VALIDATION_ERROR, 422 RESOURCE_CONFLICT (negative stock).

Side effects: Stock updated, audit log.

Idempotency: Yes (with Idempotency-Key).

POS

ID	Method	Endpoint	Purpose	Auth	Permission
POS-001	POST	/pos/sales	Create sale	Yes	pos.sales.create
POS-002	POST	/pos/sales/:id/void	Void sale	Yes	pos.sales.void

POS-001: POST /pos/sales

Purpose: Record a POS sale.

Request:

```
{
  "branchId": "b1",
  "items": [
    { "productId": "p1", "quantity": 2, "unitPrice": 50000, "discount": 0 }
  ],
  "payments": [
    { "method": "cash", "amount": 100000 }
  ]
}
```

Success (201):

```
{
  "data": {
    "id": "sale1",
    "branchId": "b1",
    "total": 100000,
    "status": "paid",
    "createdAt": "2026-09-28T22:30:00Z"
  },
  "meta": { "requestId": "req_123" }
}
```

Errors: 400 VALIDATION_ERROR, 422 RESOURCE_CONFLICT (out of stock).

Side effects: Stock reduced, audit log.

Idempotency: Yes.

Orders (WhatsApp)

ID	Method	Endpoint	Purpose	Auth	Permission
ORD-001	GET	/orders/whatsapp	List WA orders	Yes	orders.whatsapp.read
ORD-002	POST	/orders/whatsapp	Create WA order	Yes	orders.whatsapp.create
ORD-003	GET	/orders/whatsapp/:id	Get WA order	Yes	orders.whatsapp.read

Payments

ID	Method	Endpoint	Purpose	Auth	Permission
PAY-001	POST	/payments/links	Create payment link	Yes	payments.create
PAY-002	POST	/webhooks/paystack	Paystack webhook	No	Public (HMAC)

Reports

ID	Method	Endpoint	Purpose	Auth	Permission
REP-001	GET	/reports/sales	Sales report	Yes	reports.sales.read
REP-002	GET	/reports/stock	Stock report	Yes	reports.stock.read

Settings

ID	Method	Endpoint	Purpose	Auth	Permission
SET-001	GET	/settings/whatsapp	Get WA settings	Yes	settings.whatsapp.read
SET-002	PATCH	/settings/whatsapp	Update WA settings	Yes	settings.whatsapp.update

43. API Examples

Create Product

Request:

```
POST /api/v1/products
Authorization: Bearer <token>
Content-Type: application/json

{
  "sku": "SKU123",
  "name": "Example Product",
  "costPrice": 30000,
  "sellPrice": 50000,
  "taxRate": 7.5,
  "barcode": "1234567890"
}
```

Response (201):

```
{
  "data": {
    "id": "p1",
    "sku": "SKU123",
    "name": "Example Product",
    "costPrice": 30000,
    "sellPrice": 50000,
    "taxRate": 7.5,
    "barcode": "1234567890",
    "createdAt": "2026-09-28T22:30:00Z"
  },
  "meta": { "requestId": "req_123" }
}
```

Error (400):

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request.",
    "details": { "sku": "SKU already exists" },
    "requestId": "req_123"
  }
}
```

44. API Traceability Matrix

API ID	PRD Requirement	Resource	Frontend Consumer	Backend Module	Permission	Test
AUTH-002	Auth login	User	Login page	auth	Public	auth.test.ts
INV-005	Stock adjustment	StockMovement	Inventory UI	inventory	inventory.stock.adjust	inventory.test.ts
POS-001	Create sale	SalesOrder	POS UI	pos	pos.sales.create	pos.test.ts
PAY-001	Payment link	PaymentIntent	Orders UI	payments	payments.create	payments.test.ts

45. Open Decisions

- **Rate limit values:** OPEN DECISION (configure per env).
- **File upload max size:** OPEN DECISION (e.g., 5MB).
- **Audit API exposure:** OPEN DECISION (future).
- **Bulk operations:** OPEN DECISION (Phase 2).

46. API Quality Checklist

- ☐ No duplicate endpoints.
- ☐ Consistent naming.
- ☐ Consistent response structures.
- ☐ Consistent errors.
- ☐ Authorization on all protected endpoints.
- ☐ Validation on all inputs.
- ☐ Pagination on collections.
- ☐ Tenant isolation enforced.
- ☐ Idempotency where required.
- ☐ Concurrency handling for stock/sales.
- ☐ No unsafe data exposure.
- ☐ No silent breaking changes.
- ☐ No unnecessary complexity.

End of api-contract.md